HW2: Actor-Critic Method in BipedalWalker

PKU Reinforcement Learning Course

MAK CHAK WING UID: 2501213361

2026

Contents

1 Environment & Problem Setting **2**

2 PPO Method **2**

2.1 Policy Gradient with Clipping . . . 2

2.2 Generalized Advantage Estimation . . . 3

2.3 Full Objective . . . 3

2.4 Continuous Action Policy . . . 3

3 Implementation Details & Hyperparameters **4**

4 Training Results **5**

5 Evaluation Results **6**

5.1 Measurement Phase . . . 6

5.2 Video Phase . . . 6

6 Discussion **7**

7 Conclusion **7**

8 How to Run **8**

8.1 Prerequisites . . . 8

8.2 Install Dependencies . . . 8

8.3 Train and Evaluate (Full Pipeline) . . . 8

8.4 Evaluate Only (Requires Pre-trained Weights) . . . 8

1 Environment & Problem Setting

BipedalWalker-v3 is a continuous-state, continuous-action locomotion task from the Gymnasium Box2D suite. The goal is to make a 4-joint bipedal robot walk forward as far as possible without falling.

State space (24-dimensional, continuous):

- Hull angle, angular velocity, horizontal and vertical velocity
- Joint positions and angular speeds for both hip and knee joints (4 joints total)
- Two binary ground-contact indicators (left leg, right leg)
- 10 lidar rangefinder measurements

Action space (4-dimensional continuous, range $[-1, 1]$):

Each dimension controls the motor speed applied at one joint (hip and knee for each leg).

Reward structure. The agent is rewarded for moving forward, with a total possible reward exceeding 300 for reaching the far end. Falling incurs a penalty of -100. Applying motor torque costs a small amount of points. The task is considered solved at an average episode reward of 300.

Since both the state and action spaces are continuous, tabular methods are not applicable. We use Proximal Policy Optimization (PPO), an on-policy Actor-Critic algorithm that directly learns a stochastic continuous-action policy.

2 PPO Method

PPO is an Actor-Critic algorithm that maintains two networks: an **Actor** that outputs a policy $\pi_\theta(a \mid s)$, and a **Critic** that estimates the state value $V_\phi(s)$. The key innovation of PPO is a clipped surrogate objective that prevents large policy updates, improving stability over vanilla policy gradient methods.

2.1 Policy Gradient with Clipping

Let $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ be the probability ratio between the new and old policy. The clipped surrogate objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\right]$$

where $\hat{A}_t$ is the estimated advantage and ε is the clip range. The clip prevents the policy from changing too far from the old policy in a single update.

2.2 Generalized Advantage Estimation

Advantages are computed using GAE:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

where λ is the GAE smoothing parameter. GAE interpolates between the high-bias one-step TD error ($\lambda = 0$) and the high-variance Monte Carlo return ($\lambda = 1$).

Gymnasium distinguishes **terminated** (the MDP reached an absorbing state, e.g. the walker fell) from **truncated** (the time limit of 1600 steps was hit). Only true terminations zero out the bootstrap: $V(s') = 0$. On time-limit truncation the episode is cut artificially, so the return should still be bootstrapped from $V(s')$. We handle this by folding $\gamma V(s')$ into the stored reward at truncation steps before calling the GAE computation, so the delta formula requires no structural changes.

2.3 Full Objective

The total loss minimized at each update is:

$$\mathcal{L} = -\mathcal{L}^{\text{CLIP}} + c_v \mathcal{L}^{\text{VF}} - c_e \mathcal{S}[\pi_\theta]$$

where $\mathcal{L}^{\text{VF}} = \mathbb{E}_t[(V_\phi(s_t) - \hat{R}_t)^2]$ is the MSE value loss and $\mathcal{S}[\pi_\theta]$ is the entropy bonus that encourages exploration.

2.4 Continuous Action Policy

The Actor outputs the **mean** $\mu(s)$ of a diagonal Gaussian distribution, while the **log standard deviation** is a learned global parameter (not state-dependent). A tanh activation is applied to the mean output to bias it into $[-1, 1]$. Actions are sampled from $\mathcal{N}(\mu(s), \sigma^2 I)$; the raw sample and its log-probability are stored in the rollout buffer, and the sample is clipped to $[-1, 1]$ only when passed to the environment. This keeps the importance-sampling ratio $r_t(\theta)$ consistent with the executed action.

3 Implementation Details & Hyperparameters

Table 1: Hyperparameters used for PPO on BipedalWalker-v3.

Parameter	Value
Environment	BipedalWalker-v3
Algorithm	PPO
Total timesteps	2,000,000
Rollout steps per update	2,048
PPO update epochs per rollout	10
Mini-batch size	64
Optimizer	Adam
Learning rate α	3×10^{-4}
Discount factor γ	0.99
GAE λ	0.95
PPO clip range ε	0.2
Value loss coefficient c_v	0.5
Entropy bonus coefficient c_e	0.01
Gradient clipping (ℓ_2 norm)	0.5
Network hidden dim	256
Random seed	42

Network architecture. Both the Actor and Critic use a two-layer MLP:

$$\text{Input}(24) \;\rightarrow\; \text{FC}(256) \;\rightarrow\; \text{ReLU} \;\rightarrow\; \text{FC}(256) \;\rightarrow\; \text{ReLU} \;\rightarrow\; \text{Output}$$

The Actor output head applies tanh to produce the action mean. A single shared Adam optimizer is used for both networks.

Advantages are normalized per minibatch to zero mean and unit variance, which is critical for training stability in PPO.

4 Training Results

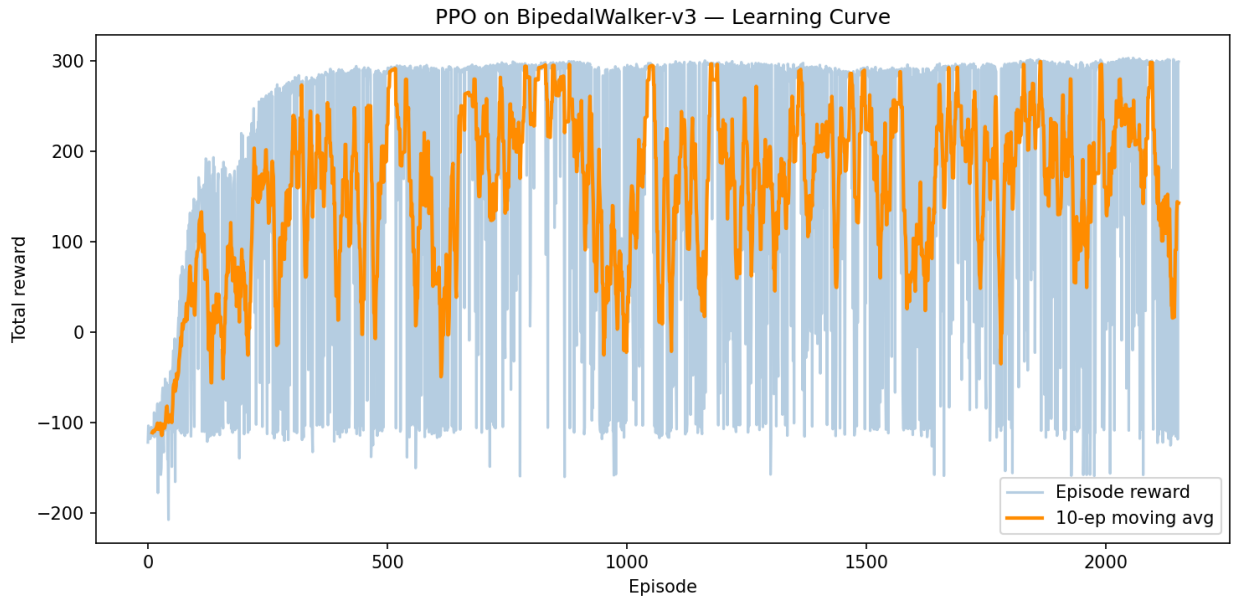

Figure 1: PPO learning curve on BipedalWalker-v3. Blue: raw episode reward per episode. Orange: 10-episode moving average. Training ran for 2,000,000 timesteps over 2,153 episodes.

The agent begins with near-random behavior (average reward around −110), corresponding to the walker falling immediately. From around episode 100–200, the policy starts improving as the rollout buffer accumulates more diverse transitions and the entropy bonus promotes exploration.

The 10-episode moving average rises steadily, reaching its peak of approximately **299.5** at around step 1,740,000 (episode 1,864). This is near the task's solve threshold of 300. However, as training continues past the peak, the average reward regresses to approximately **143** by the final step. This late-stage regression is a well-known behavior of PPO on this environment: without a learning rate schedule or early stopping, the policy can drift away from its best configuration after the initial convergence.

5 Evaluation Results

After training, the saved policy is evaluated in two phases using deterministic rollouts (action = policy mean, no sampling noise).

5.1 Measurement Phase

To obtain a statistically reliable estimate, the policy is evaluated over **20 episodes** with seeds offset from the training seed (seeds 1042–1061). Results:

Table 2: 20-episode measurement statistics. Deterministic rollouts, seeds independent of training.

Metric	Value
Mean reward	**252.27**
Std deviation	112.96
Min / Max	−45.59 / 309.10
Success rate (reward $\geq$ 200)	**16 / 20**

The policy succeeds (reward $\geq$ 200) in 16 of 20 episodes, with a mean reward of 252.27. The high standard deviation reflects the bimodal nature of this task: episodes end either near 300 (successful walk) or near −100 (early fall), with few intermediate outcomes.

5.2 Video Phase

Five additional episodes were recorded for the submission video (seeds 42–46); four achieved rewards above 300. Episodes 1–3 were concatenated into a single ~59-second submission video using ffmpeg.

The learned gait achieves high reward through a characteristic one-leg hopping strategy, where the robot primarily uses one leg to push forward while the other serves for balance. Although this is not the intended bipedal walking gait, it is a valid local optimum in this environment, the reward function incentivizes forward progress, which hopping can achieve efficiently.

6 Discussion

Local optimum in gait. The one-leg hopping behavior is a known local optimum for PPO on BipedalWalker-v3 with a single training environment. The agent finds this strategy early in training and it is sufficient to score close to 300, so the policy gradient has little incentive to explore further. Parallel environments (e.g., 32 vectorized envs as in the RL Baselines3 Zoo configuration) provide more diverse experience and help the agent discover proper bipedal gaits.

Potential improvements. Several directions could improve performance or gait quality:

- **Learning rate decay** — A linearly or exponentially decaying learning rate would help stabilize the policy near its peak reward and prevent the late-stage regression observed in the training curve.
- **Vectorized environments** — Training with multiple parallel environments increases sample diversity per rollout, reduces the likelihood of converging to a single-leg gait, and accelerates wall-clock training time.
- **Observation normalization** — A running mean/variance normalizer applied to the 24-dimensional observation vector can help the network learn uniformly from all observation dimensions, as their natural scales differ significantly.

7 Conclusion

This homework implements PPO on the BipedalWalker-v3 continuous control task. The key observations are:

- **PPO is well-suited for continuous action spaces.** The Gaussian policy with learned log-std and clipped surrogate objective provides stable on-policy learning without the need for a replay buffer.
- **GAE is important for advantage quality.** It reduces variance relative to Monte Carlo returns while keeping bias manageable, which is especially useful in long-horizon locomotion tasks.
- **Entropy bonus prevents premature collapse.** Keeping $c_e = 0.01$ maintained exploration throughout training. Removing it entirely (setting $c_e = 0.0$) caused the policy to collapse to always-falling within 1–2M steps.
- **The agent reaches near-solved performance** (peak 10-ep average of ~299.5) within 2M timesteps, demonstrating that PPO with standard hyperparameters is competitive on this task.

8 How to Run

8.1 Prerequisites

- Python 3.10+

8.2 Install Dependencies

```
pip install "gymnasium[box2d]" "gymnasium[other]" matplotlib numpy torch
```

8.3 Train and Evaluate (Full Pipeline)

```
python code.py
```

This will:

1. Train the PPO agent for 2,000,000 timesteps
2. Save the trained model to `agent.pth`
3. Plot the reward curve to `reward_curve.png`
4. Record 5 evaluation episodes to `video/`

8.4 Evaluate Only (Requires Pre-trained Weights)

```
python code.py --eval-only
```

Loads `agent.pth` from a prior training run. Runs a **20-episode measurement phase** (no video) and prints mean ± std, min/max, and success rate, then records 5 video episodes to `video/`.